Understanding graph representation learning in reinforcement learning based logic synthesis

Anonymous Author(s)
Affiliation
Address
email

Abstract

Computer chips ought to be as small as possible for cost efficiency. The problem of minimizing chip size while preserving the chip's functionality has been studied through the prism of and-inverter graphs (AIGs), since any Boolean function can be represented by an AIG. Because the complexity of AIG size minimization is unknown—it is an instance of the Minimum Circuit Size Problem, which lies in NP but is not known to be NP-hard—no procedure returns optimal AIGs at the sizes of interest, and there is therefore no supervision signal in the form of optimal AIGs to imitate. Reinforcement learning, and more generally planning, have been used to optimize AIGs in the absence of such a signal. Contemporarily, graph representation learning has been applied to train machine learning models to perform tasks on graph data. More recently, those two technologies have been combined and reported to outperform state-of-the-art logic synthesis heuristics, but the relative importance of graph representation learning in those methods remains unknown. Is graph representation learning useful when optimizing and-inverter graphs with reinforcement learning?

1 Introduction

A circuit that computes a given Boolean function with fewer logic gates occupies less area and is cheaper to manufacture [31]. Given a Boolean function, logic synthesis methods [] are used to find a small circuit that computes it. Because every Boolean function can be represented with AND and NOT gates only, logic synthesis methods represent circuits with and-inverter graphs (AIGs) which nodes are AND gates and edges can invert incoming bits.

However, finding the smallest AIG encoding a given Boolean function is an open problem. It is an instance of the Minimum Circuit Size Problem, which lies in NP but is not known to be NP-hard [16], and which is related to Graph Isomorphism [2]. In practice, AIGs are reduced by applying sequences of heuristics that rewrite the graph while preserving functional equivalence []. Many such heuristics are implemented in the `ABC` [5] software. Choosing the best heuristics sequence is a search problem.

In the absence of exact solvers, machine learning such as supervised learning has been applied to AIG minimization [39]. However it suffered from two major weaknesses: first, the supervision signal to train from could be a suboptimal minimal AIG since no exact solver exists, and second, graph generative models are still quite limited []. Successful machine learning approaches are unsupervised ones that only require a scalar signal (often the size of a given AIG which is cheap to obtain): Bayesian optimization [11], multi-armed bandits [30], reinforcement learning (RL) [15, 51] and Monte Carlo tree search [33]. Contemporarily, graph representation learning has been applied to train models on graph data, and circuits are graphs: task-agnostic encoders built specifically for AIGs are now available [19, 22, 49, 50]. The two lines of work have since been combined. Several of the methods above encode the current AIG with a graph neural network rather than with a fixed set of summary

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

statistics [51, 33, 3], and achieve reductions beyond the expert ABC heuristics. The motivation is the same as the one behind the very first deep RL algorithm, which was designed specifically for Backgammon [42]: the state space of AIG minimization is huge and cannot be enumerated, so the value of a state has to be generalized from other states. A learned representation of the circuit should carry structural information that statistics discard, and that is what should allow a deep reinforcement learning algorithm [37] to generalize to unseen circuits: "I have seen a similar graph before, I should probably edit the AIG in a similar way to get good results".

What those RL methods do not report is how much of their performance is attributable to the learned representation design. We are not aware of a study that isolates the contribution of the AIG encoder to RL training. Our work reports such an ablation, in the fixed-horizon setting that Zhu et al. [51] introduced and that later work reuses. We keep that pipeline fixed—the Markov decision process, the policy and value heads, the training algorithm and the hyper-parameter grid—and vary only how the AIG is presented to the policy. We show the following results.

Contributions: First, we show that adding a graph convolutional network to the policy does not improve AIG reduction beyond a multi-layer perceptron that reads graph statistics (number of nodes, of levels, . . .). Second, we derive empirical performance bounds–in terms of AIG value prediction–for message passing networks bounded by the 1-dimensional Weisfeiler-Leman refinement as well as for AIGs specific models.

2 Related work

2.1 Combinational optimization

Combinational synthesis is a class of fast algorithms that reduce an AIG by rewriting it locally, and that work well in practice [24, 25, 7]. `ABC` [5] is the reference implementation and is widely used in academia. Throughout this paper we call *primitives* its standard local transformations, `balance`, `rewrite`, `resub` and `refactor`, and *heuristics* the named sequences of primitives it ships, such as `resyn2`. For instance, the primitive `refactor` performs iterative collapsing and refactoring of logic cones in the AIG, attempting to reduce both the number of nodes and the number of levels. Since no primitive is globally optimal, reduction is done by composing them into heuristics. Hand-designed heuristics are the reference points against which learned methods are measured: `resyn` [32] applies `balance`, `rewrite`, `balance`, `rewrite`, `balance`, and the `ABC` documentation also ships a more aggressive heuristic known as the "lazy man's synthesis" [48]. Searching over sequences of primitives has been automated without learned circuit representations, with Bayesian optimization [11] and with multi-armed bandits [30].

2.2 Graph representation learning for AIGs

A separate line of work learns representations of circuits that are meant to transfer across circuits, rather than a policy for one circuit. DeepGate4 [50] is representative: it learns node embeddings of AIGs and scales them to large designs. Three encoders from this line—DeepGate [19], PolarGate [22] and FuncGNN [49]—are among the models we evaluate in Section 5.2. `GraphBench` [39] is, to our knowledge, the first AIG reduction benchmark built for methods that must perform on unseen AIGs. It provides a training set of Boolean functions of varying input and output dimension and a disjoint test set, and it reports node reduction relative to the expert `ABC` heuristics `resyn2`, `resyn2`, `dc2`, `resyn2rs`, `resyn2`, applied in that order. Its main result is negative: given the AIG obtained by the naive construction from a truth table, specialized DAG generative models [6, 20] do not preserve functional equivalence while reducing size. Our results are negative in the same sense, for a different family of models.

2.3 Reinforcement learning for AIGs

The works discussed in this section agree on the action space, which is the one drawn in Figure 1: a single `ABC` primitive applied to the whole graph, never an entire heuristic such as `resyn2`, and finer than a primitive only in Haaswijk et al. [12]. Section 6 returns to what that choice gives up. What varies is the encoding of the circuit. Most recent methods, here and in the neighboring chip design tasks of Section 2.4, run a graph neural network on the circuit as part of the deep reinforcement learning agent, sometimes alongside engineered features and sometimes as the whole state representation [12, 51, 33, 3, 23, 44], while others describe the circuit with graph statistics alone [15, 21]. None of them reports how much of its performance is due to the graph neural network, so the importance of the learned representation is unknown. This is precisely the axis this paper ablates.

Hosny et al. [15] formulate synthesis as an MDP whose states are vectors of AIG statistics—numbers of primary inputs and outputs, nodes, edges, levels and latches, and the proportions of AND and NOT nodes—whose actions are `ABC` primitives, and whose reward combines node count and level count. Zhu et al. [51] keep `ABC` primitives as actions but encode the whole AIG with a graph convolutional network, alongside a small vector of statistics and recent actions, and reward the relative reduction in node count. Episodes there have a fixed horizon and no early stopping, so the induced problem is a search over flows of fixed length. This is the formulation we adopt, horizon included, and we state it in full in Section 4.1. Haaswijk et al. [12] predate both and act with Boolean algebraic operations on majority-inverter graphs; we do not compare against it directly because MIGs and AIGs are not directly comparable. Both Hosny et al. [15] and Zhu et al. [51] train with policy gradient methods [41, 46, 28]. We are not aware of a value-based [45, 27] treatment, even though AIG editing is deterministic; Section 6 returns to what that omission means for our conclusion. These methods report reductions beyond the `ABC` heuristics, but they are not designed to generalize: a policy is trained to reduce one AIG, so the training cost is paid again for every new circuit. Search-based methods lift that restriction. AlphaSyn [33] and AlphaChip [23] plan online and therefore apply to circuits unseen at training time, but there is no separation between training and inference: every edit requires a lookahead, and every lookahead step requires network forward passes. ABC-RL [3] sits in between: a policy pre-trained on other circuits guides the tree search, and its influence is scaled down when the circuit at hand is far from anything in the training set. Two further formulations widen the setting rather than the state: ESE [35] adds technology mapping operations to the action space and trains with PPO [37], and CB-EVO [21] drops the sequential state altogether and treats the choice of primitive as a contextual bandit whose context is a set of circuit features.

2.4 Graph reinforcement learning for other combinatorial optimization

AIG reduction is an instance of graph structure optimization in the taxonomy of Darvariu et al. [8], who survey reinforcement learning applied to combinatorial problems on graphs and report that a graph neural network encoder is the common choice of state representation. Within chip design the same combination is used for transistor sizing [44] and for macro placement [23]; in both cases the encoder is motivated as the component that enables transfer to new circuits.

3 Background material

3.1 And-inverter graphs minimization

Given an arbitrary Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, the objective is to find an And-Inverter Graph that encodes f with as few nodes as possible.

Definition 1 (And-Inverter Graph, following section 3.2.1 from Stoll et al. [39]) *An And-Inverter Graph $\mathcal{G} := (V, E)$ is a directed acyclic graph of in-degree (fanin) at most two. The node set is partitioned into input nodes of in-degree 0, $V^{in} := \{v_1^{in}, \dots, v_n^{in}\}$, AND nodes of in-degree two, $V^{AND} := \{v_1^{AND}, \dots, v_k^{AND}\}$, and output nodes of in-degree one, $V^{out} := \{v_1^{out}, \dots, v_m^{out}\}$; AIGs are therefore heterogeneous graphs [38]. The edge set is $E \subseteq V \times V \times \{0,1\}$, with 0 indicating direct transmission and 1 indicating input negation (NOT). For an input $\boldsymbol{x} \in \{0,1\}^n$, values are assigned to input nodes, propagated through gate nodes (applying inversion where specified), and collected at output nodes to yield $\mathcal{G}(\boldsymbol{x}) \in \{0,1\}^m$. Because the combination of AND and NOT is functionally complete, any Boolean function can be represented using only these two operations.*

We write $\mathcal{G}_f := \{\mathcal{G} : \mathcal{G}(\boldsymbol{x}) = f(\boldsymbol{x}) \; \forall \boldsymbol{x} \in \{0,1\}^n\}$ for the set of AIGs functionally equivalent to f. AIGs are non-canonical representations of Boolean functions [26]—two different AIGs can represent the same function—so $|\mathcal{G}_f| > 1$ in general: the first two graphs of Figure 1 are functionally equivalent and have eight and seven nodes respectively.

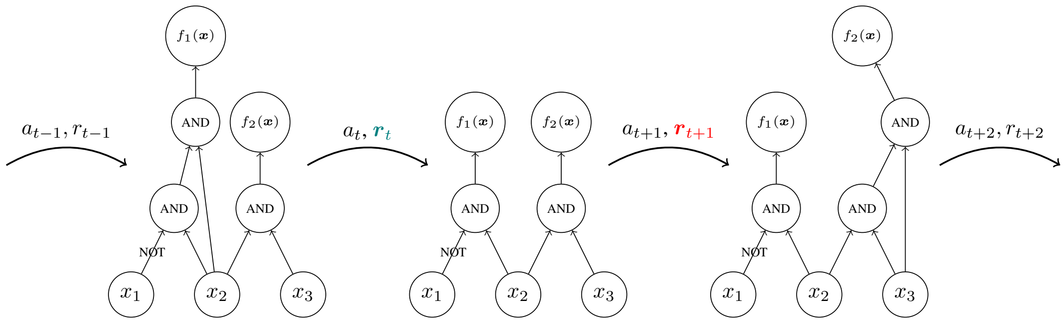

Figure 1: A trajectory in a generic MDP for AIG reduction, on AIGs of the Boolean function $f(x_1, x_2, x_3) = (\bar{x_1} \wedge x_2,\ x_2 \wedge x_3)$. Every method surveyed in Section 2 acts at this granularity: each action is one `ABC` primitive applied to the whole graph, and the primitives of Section 2.1 preserve functional equivalence by construction, so every state reachable from $\mathcal{G}_0$ still computes f.

Definition 2 (And-Inverter Graph reduction) *Given a Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, we want to find*

$$\mathcal{G}^\star \subseteq \underset{\mathcal{G} \in \mathcal{G}_f}{\arg\min} |V|,$$

the AIGs functionally equivalent to f with as few nodes as possible.

The hardness of this problem is not known. It is an instance of the Minimum Circuit Size Problem [16], a natural and well-studied problem in NP that is not known to be NP-hard[1], and which is related to Graph Isomorphism [2]. In the absence of a breakthrough in complexity theory, this motivates the development of the approximate algorithms of Section 2.1.

3.2 Markov decision process

AIG reduction can be formulated as a sequential decision making task in which the environment is the current AIG, the actions are edits of that AIG, and the rewards are shaped to favor edits that remove nodes. Formally, this is the problem of finding an optimal policy for a Markov decision process [4, 34].

Definition 3 (Markov decision process) *An MDP is a tuple $\mathcal{M} = \langle S, A, R, T, T_0 \rangle$. S is a finite set of states representing all possible configurations of the environment (e.g. AIGs, or AIG statistics). A is a finite set of actions available to the agent (e.g. AIG edits). $R : S \times A \to \mathbb{R}$ is a deterministic reward function that assigns a real-valued reward to each state-action pair (e.g. number of deleted nodes). $T : S \times A \to \Delta(S)$ is the transition function that maps state-action pairs to probability distributions over next states $\Delta(S)$ (e.g. how graph edits change the AIG; here the transitions are deterministic). $T_0 \in \Delta(S)$ is the initial distribution over states (e.g. a set of AIGs to reduce).*

Definition 4 (Reinforcement learning objective) *Given an MDP $\mathcal{M} \equiv \langle S, A, R, T, T_0 \rangle$, the goal is to find a policy $\pi : S \to A$ that maximizes the expected discounted sum of rewards*

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 \sim T_0, s_{t+1} \sim T(s_t, \pi(s_t))\right]$$

where $0 < \gamma \leq 1$ is the discount factor that controls the trade-off between immediate and future rewards.

We write $V^\star(s)$ for the value of a state under an optimal policy, i.e. the discounted sum of rewards collected from s by acting optimally. $V^\star$ is the quantity a critic is trained to approximate in actor-critic methods, and the quantity a state representation must be able to distinguish states by; Section 5.2 uses it to measure what a representation can express, independently of any training procedure. Reinforcement learning algorithms optimize the objective of Definition 4 by trial and error; we refer to Sutton and Barto [40] for an introduction.

[1] `https://mcsp.work/index.html`

3.3 Message passing neural networks

Every graph encoder considered in this paper belongs to the message passing family [10]. A message passing neural network attaches a vector $h_v^{(0)}$ to each node v, initialized from that node's features, and refines it over k rounds:

$$h_v^{(t+1)} = \phi^{(t)}\Big(h_v^{(t)}, \textstyle\bigoplus_{u \in \mathcal{N}(v)} \psi^{(t)}\big(h_v^{(t)}, h_u^{(t)}, e_{uv}\big)\Big),$$

where $\mathcal{N}(v)$ are the neighbours of v, e_{uv} is an optional label on the edge between u and v, $\bigoplus$ is a permutation-invariant aggregator, and $\phi^{(t)}, \psi^{(t)}$ are learned. A graph-level vector is then obtained by pooling $\{h_v^{(k)}\}_{v \in V}$ with a second permutation-invariant readout. Architectures in this family differ only in the choice of ψ, $\bigoplus$ and ϕ: GCN [18], GIN [47], GAT [43] and GraphSAGE [14] are instances, and so are the AIG-specific encoders we evaluate in Section 5.2.3. They also share a limitation. After k rounds, two nodes that k iterations of the 1-dimensional Weisfeiler-Leman refinement assign the same color carry the same embedding [47, 29], so no network in the family can tell apart two AIGs that 1-WL cannot. Section 5.2.2 turns this into a lower bound on the error of every such encoder.

4 Methodology

We take the pipeline of Zhu et al. [51]—the MLCAD 2020 formulation that later work reuses—and vary only how the current AIG is presented to the policy. Two things differ from their setup, and both make a useful encoder easier to detect rather than harder: we train with PPO [37] rather than REINFORCE [46], and we tune every encoder over a common hyper-parameter grid instead of reporting a single configuration. Everything else is held fixed across the runs we compare, so the encoder is the only difference between two of them.

4.1 The Markov decision process

We take the MDP of Zhu et al. [51] unchanged, instantiating Definition 3 as follows. Let $\mathcal{G}_0$ be the AIG to reduce and write n_t and ℓ_t for its number of AND nodes and of levels after t actions. The five actions are the `ABC` primitives {`balance`, `rewrite`, `rewrite -z`, `refactor`, `refactor -z`}, where `-z` enables zero-cost replacements; transitions are deterministic and every primitive preserves functional equivalence, so every reachable state lies in $\mathcal{G}_f$ and the problem of Definition 2 is never violated, only under-solved. Episodes start from $\mathcal{G}_0$ and last exactly $H = 20$ actions with no early stopping, so the induced optimization is a search over flows of length 20 from a five-element alphabet. The reward $R(s_t, a_t) = 1 - n_{t+1}/n_0$ is issued at every step and is a level rather than a difference, so with $\gamma = 1$ the return is maximized by an agent that keeps the AIG small throughout the episode and not merely at the end. Levels do not enter it: unlike the multi-objective reward of Hosny et al. [15] we optimize node count alone, which is the objective of Definition 2. The agent sees the state through a vector $x_{\mathcal{G}}$, always present (Section 4.2), and optionally a graph, which is what the encoders under comparison do or do not consume (Section 4.3). Our environment also implements a seven-action variant adding `resub` and `resub -z`, and the full `ABC` action space, neither of which is used below.

4.2 Engineered graph features

The vector part of the observation, $x_{\mathcal{G}} \in \mathbb{R}^{|A|+5}$, summarizes the current AIG and the recent history of the episode: $x_{\mathcal{G}} = \Big[\underbrace{\tfrac{1}{q}\sum_{j=1}^{q} \mathbf{1}[a_{t-j} = a]}_{a \in A},\ \tfrac{n_t}{n_0},\ \tfrac{\ell_t}{\ell_0},\ \tfrac{n_{t-1}}{n_0},\ \tfrac{\ell_{t-1}}{\ell_0},\ \tfrac{t}{H}\Big]$. The first $|A| = 5$ entries are a normalized histogram of the last q actions, with $q = H$, so they summarize the whole flow applied so far; the next four are the AND and level counts of the current and of the previous AIG, each divided by its value in $\mathcal{G}_0$ so that the vector is scale-free across benchmarks of very different sizes; the last is the normalized time step, which makes the finite-horizon problem Markov in the observation. It is computed in a single pass over the graph and adds no parameters of its own, and it carries no structural information about the circuit: two AIGs with the same size, depth and history are indistinguishable to it.

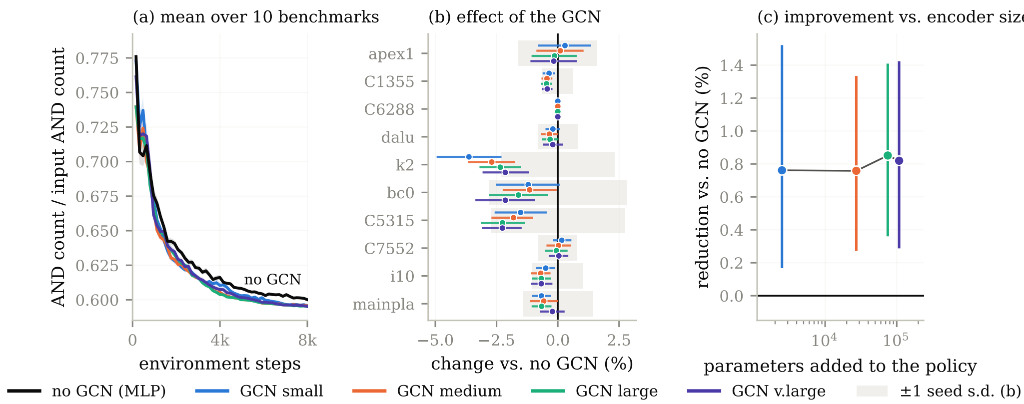

Figure 2: PPO on the ten MLCAD benchmarks with five observation encoders, hyper-parameter matched. (a) mean AND count over the ten benchmarks; (b) per-benchmark change against the MLP, shaded band is one seed standard deviation; (c) mean reduction against the parameters the encoder adds to the policy.

4.3 Graph convolutional networks

The graph part of the observation is the AIG itself, as in Zhu et al. [51]: nodes carry a one-hot encoding of their type among the six types `abc_py` exposes, a directed edge runs from each fanin to the node it feeds, and edges carry no label, so complementation is visible to the encoder only through node types. A graph convolutional network [18] runs a fixed number of the message passing rounds of Section 3.3 over it and pools the node embeddings into a graph-level vector, concatenated with x_G before the policy and value heads. We instantiate four such encoders, from $2{,}496$ to $107{,}968$ parameters added to the policy: three at depth three with widths from 12 to 128, and one at depth five, so that depth and width are not confounded (Table 1 in Appendix A.1). The baseline drops the graph part, so that the policy reads x_G alone through a multi-layer perceptron: the encoder is only ever added to the baseline, never substituted for it, and the comparison is between x_G and x_G augmented with a learned representation of the circuit.

4.4 Reinforcement learning

Policies are trained with PPO [37] on the MDP of Section 4.1, rather than with the REINFORCE of Zhu et al. [51], so that a failure of the graph encoder cannot be attributed to a weak learner. The resulting procedure, which we call GraphPPO and state in full as Algorithm 1 in Appendix A.1, is ordinary PPO with the encoder made explicit: the encoding of the current AIG is the only place where the five configurations we compare differ, and the encoder is trained end to end with the policy and value heads rather than pre-trained or frozen. We use the implementation of `stable-baselines3` [36] with $\gamma = 1$, the horizon being finite and fixed, and train for $8{,}000$ environment steps, that is $N = 50$ updates; after every update we evaluate the greedy policy for one episode, which is the quantity the learning curves report. Each encoder is then tuned over the same grid of learning rates, entropy coefficients and seeds, and encoders are compared only on the grid cells that all of them completed, so that the comparison is between encoders and not between differently tuned systems. Appendix A.1 gives the remaining hyper-parameters.

5 Experiments

5.1 Graph reinforcement learning

Setup. The benchmarks are the ten MLCAD circuits `apex1`, `bc0`, `C1355`, `C5315`, `C6288`, `C7552`, `dalu`, `i10`, `k2` and `mainpla`. Each of the five encoders is run on each benchmark over a common grid of three learning rates, three entropy coefficients and five seeds, for $2{,}250$ runs in total, and every number below averages over the 424 of 450 (benchmark, learning rate, entropy, seed) cells that all five encoders completed. No encoder is therefore given its own best hyper-parameters — tuning each separately would let the selection step, rather than the encoder, decide the winner. Appendix A.1 gives the grid and the runs we drop. We report the final AND count divided by the AND count of the input AIG, so that benchmarks of different sizes can be averaged.

Adding a GCN to the policy does not improve AIG minimization (Figure 2). Every GCN reduces the final AND count by 0.76–0.85% relative to the MLP, which is less than the 1.23% standard deviation induced by changing the seed alone, and a $43\times$ increase in encoder size moves that figure by nothing. The encoder is the only component that differs between these runs, so what the comparison shows is that we cannot separate its effect from the noise of reseeding. Figure 8 in Appendix D breaks the same runs down by benchmark.

Appendix B repeats the comparison on `ex200`–`ex299`, a harder suite of larger and more varied circuits; the result appears to carry over, with the same difference of well under one percent and no growth with circuit size.

5.2 Value prediction

The previous section compares encoders inside one algorithm, so a null result there could be a property of that algorithm rather than of the representations. We therefore ask a question that does not involve a training algorithm at all: how well can the optimal value $V^\star$ of a state be predicted from what a given class of models is able to see?

5.2.1 Building a dataset

Computing $V^\star$ exactly is out of reach: the tree of flows over the five primitives of Section 4.1 has 5^H leaves, and every edge of it costs one `ABC` call on the full circuit. We therefore build, for each benchmark, a dataset of states paired with a Monte-Carlo estimate of their value, obtained by sampling both the states and the continuations from them.

States. A state is the AIG obtained by applying a uniformly random flow to the benchmark $\mathcal{G}_0$: we draw a length $L \sim \mathcal{U}\{2, \ldots, 10\}$, then L primitives independently and uniformly from the five-element action set, and run them in order. We sample $1{,}000$ such states per benchmark. Each one is written to disk as an `.aig` file and stored together with the statistics x_G of Section 4.2 recorded at that point of the flow, so that every model class of Sections 5.2.2 and 5.2.3 is evaluated on exactly the same states and the same targets—the graph models read the `.aig` file, the MLP reads x_G.

Targets. From each stored state s we run $M = 100$ independent rollouts of 8 further uniformly random primitives, reload the state between rollouts, and record the AND count $n^{(j)}(s)$ reached at the end of rollout j. The target is the best of them,$\widehat{V}(s) = \min_{j\leq M} n^{(j)}(s)$, that is, the smallest AND count the sampler managed to reach within 8 actions of s. This is a best-of-M estimate: it is an upper bound on $\min$ over the 5^8 flows of length eight, and it approaches it from above as M grows. Producing one dataset therefore costs $1{,}000 \times (L + M \times 8) \approx 8 \times 10^5$ `ABC` primitive calls per benchmark and per seed, which is why M is lowered to 10 on `C6288`, by far the most expensive circuit of the suite to rewrite.

What the target measures. $\widehat{V}$ is expressed in AND nodes rather than in returns, so it is ordered the other way round from the value of Definition 4—smaller is better—and it truncates the horizon at eight steps instead of the $H = 20$ of the reinforcement learning experiments. Neither affects the comparison between model classes. The map $n \mapsto 1 - n/n_0$ from AND count to the reward of Section 4.1 is affine and order-reversing, all the errors we report are ratios to the variance of the target and are therefore free of its units, and the rank correlations we report are unchanged up to sign. What the truncation does mean is that our targets approximate the value of an eight-step-to-go problem under a best-of-M estimate, not the exact $V^\star$ of the full episode, and the bounds of Section 5.2.2 are lower bounds on the error of predicting *that* quantity.

Protocol. The procedure is repeated with ten seeds (0 to 9) per benchmark, each seed drawing its own $1{,}000$ states and its own rollouts, and each resulting dataset is split at random into training, validation and test sets; the figures of this section report over those ten repetitions. The benchmarks are the same ten MLCAD circuits as in Section 5.1. Two of them are degenerate for this purpose:

the sampler reaches only six distinct values of $\widehat{V}$ on `C1355` and only two on `C6288`. We therefore exclude `C6288` and report over the remaining nine benchmarks, flagging `C1355` wherever it appears.

5.2.2 Theoretical lower bounds on the prediction error

A model that cannot distinguish two states must assign them the same prediction. For a class of models, this partitions the state space, and the best achievable squared error is attained by the predictor that outputs, for each part, the mean target over that part. That error is a lower bound: no model in the class can beat it, regardless of its size, its training procedure or its optimization. This is the protocol of `Graphtester` [1], which measures the theoretical boundary of graph neural networks on a given dataset rather than the performance of one trained instance; we reimplement it so that it applies to our four state representations and to the target of Section 5.2.1.

The classes we bound. We compute the bound for four classes, ordered by what they can see. A *size* model sees the number of nodes $|V_{\mathcal{G}}|$ alone—this is what a 0-layer message passing network reduces to, since with no rounds of message passing the readout can only count nodes. A *size and depth* model sees the pair $(|V_{\mathcal{G}}|, d_{\mathcal{G}})$, where $d_{\mathcal{G}}$ is the logic depth reported by `ABC`. An $x_{\mathcal{G}}$ model sees the engineered statistics of Section 4.2 in full, the action histogram included, which is exactly what the MLP baseline reads. A *message passing* model sees the 1-WL coloring of the AIG, which upper bounds the separating power of every architecture of Section 3.3 by the argument given there. Comparing the third and the fourth is the question of this section: what a learned encoder could express, against what the statistics already express. The two are incomparable rather than nested, and this matters for reading the result below. Only the current size and depth entries of $x_{\mathcal{G}}$ are functions of the current AIG; the previous step's size and depth, the normalized time step and the action histogram are episode history that no encoder of the current graph can recover. The $x_{\mathcal{G}}$ bound may therefore fall below the 1-WL bound without contradicting it, and a policy that reads both—which is what Algorithm 1 does—is bounded by neither alone.

Computing the partitions. For the first two classes the partition is by exact equality of the descriptors. For $x_{\mathcal{G}}$, the full feature vector is snapped to a grid of spacing 10^{-8} and states are grouped by equality of the snapped vector, so the model we bound and the MLP of Section 4.4 see the same thing. For message passing, we group states by their 1-WL graph hash, computed with `networkx`'s [13] `weisfeiler_lehman_graph_hash` at every number of iterations $k = 1, \ldots, 50$, with each node labeled by its type.[2] We bound four views of the AIG, which is what the four message passing curves of Figure 3 report: the undirected graph $\mathcal{G}$, the directed graph $\mathcal{G}^{\rightarrow}$ in which refinement follows the fanin relation, and the two variants $\mathcal{G}^{e}, \mathcal{G}^{\rightarrow e}$ that additionally label each edge with its complement bit, so that inversion is visible to the refinement rather than only through node types. Errors are divided by $\max(1, \mathrm{Var}\,\widehat{V})$; in AND-node units the variance is orders of magnitude above one on every benchmark, so the clamp never binds and the reported quantity is the plain normalized error.

Metrics. Besides the squared error we report how well each class can *order* states, since a critic that ranks successors correctly is useful even when its absolute errors are large. We measure the rank association between $\widehat{V}$ and the group-mean predictor of the class with Spearman's ρ, Kendall's τ, and the weighted τ, which puts more weight on the top of the ranking—the states an agent would actually select. Rank correlations are computed at $k = 50$ refinement iterations.

A control for partition granularity. The bound is computed in sample: the group means are fitted on the same $1{,}000$ states the error is measured on. A partition fine enough to isolate every state therefore drives it to zero regardless of whether the representation carries any signal about $\widehat{V}$. We control for this by recomputing every bound with the targets randomly permuted across states—25 permutations per dataset, 8 on `mainpla` and `C6288`—which leaves the partition intact and destroys the association. The gap between a curve and its permuted counterpart, not the height of the curve, is what indicates that a representation carries information about the value; the same control is applied to the rank correlations, where the null is the correlation between $\widehat{V}$ and a random permutation of itself.

[2] The hash is a 16-byte `blake2b` digest. A collision would merge two groups that 1-WL separates and can only raise the reported value, so the bound would become conservative rather than optimistic; at $1{,}000$ graphs per dataset the probability of one is negligible.

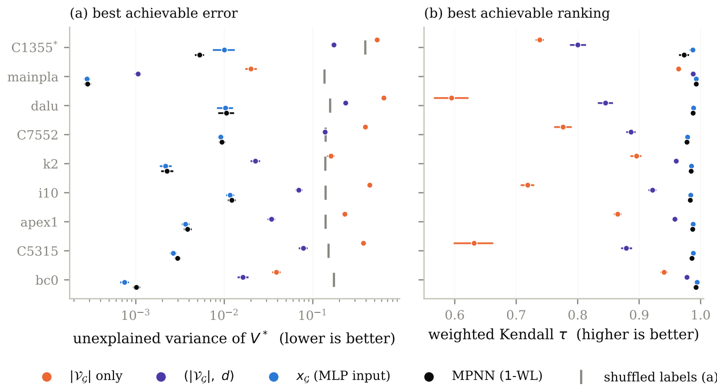

Figure 3: Best error (a) and best ranking (b) achievable by each model class on $\widehat{V}$, over nine benchmarks and ten dataset seeds. Each class is replaced by the optimal predictor for the information it can distinguish, so the value shown cannot be beaten by any model in that class. *`C1355` has only six distinct targets.

Every bound we report sits one to two orders of magnitude below its shuffled-label counterpart—between 11 and 474 times below it, depending on the benchmark—so the partitions are not merely fine enough to memorize the targets. The two partitions we compare are also of nearly equal granularity, at 865 parts for $x_{\mathcal{G}}$ and 860 for 1-WL out of $1{,}000$ states on average, so neither class is given more degrees of freedom than the other.

No message passing network can predict $\widehat{V}$ as well as the statistics the MLP already sees (Figure 3). A k-layer MPNN separates graphs at most as finely as k iterations of 1-WL, so grouping states by their 1-WL color and predicting each group's mean $\widehat{V}$ gives an error no MPNN can beat; the same construction bounds a 0-layer network, a $(\text{size}, \text{depth})$ model, and a model seeing the MLP's features $x_{\mathcal{G}}$. Message passing is worth a great deal over global descriptors: the bound falls from 0.02–0.64 of the target variance for node count alone, to 1.1×10^{-3}–0.24 once depth is added, and to 3×10^{-4}–1.2×10^{-2} for 1-WL. It is worth nothing over $x_{\mathcal{G}}$. On eight of the nine benchmarks the $x_{\mathcal{G}}$ bound is the *lower* of the two, at 0.73 to 0.99 times the MPNN bound; only `C1355` goes the other way, where message passing is better by $1.93\times$. The ordering is stable within a dataset: paired by seed, $x_{\mathcal{G}}$ is lower on all ten seeds of six benchmarks and on seven and eight of ten for `dalu` and `C7552`. For ranking the two classes are indistinguishable — weighted Kendall $\tau \geq 0.97$ for both on every benchmark, with $x_{\mathcal{G}}$ ahead by at most 0.014 — while size and depth alone reach only 0.60–0.99. This is possible because $x_{\mathcal{G}}$ is not a coarsening of the graph: it carries the episode history described above, which no encoder of the current AIG can reconstruct. Depth is irrelevant — the bound moves by at most 1.1×10^{-3} between $k = 1$ and $k = 50$, and edge direction and inverter bits change it by less than 10^{-6} (Appendix C). `C6288` is excluded: its target takes two distinct values.

Limitations of the bounds. Four restrictions should be read with the numbers above. First, they hold over the state distribution of Section 5.2.1—AIGs reached by uniformly random flows of length two to ten—and not over the states a trained policy visits, which are fewer and more correlated; a representation could be uninformative on the former and useful on the latter. Second, each dataset descends from a single benchmark, so what is bounded is the ability to tell apart states *of one circuit*, whereas the argument for learned encoders in Section 4.3 is transfer *across* circuits; our experiment does not test that claim, and a cross-circuit dataset could behave differently. Third, $\widehat{V}$ is a Monte-Carlo estimate, so part of its variance is sampling noise that no representation can predict. That noise raises every bound by roughly the same amount and therefore compresses the ratios between classes towards one, which works in favor of the negative conclusion we draw; the 1-WL and $x_{\mathcal{G}}$ bounds being close is evidence that message passing adds little, but not proof that the noise floor is not what they are both close to. Fourth, the 1-WL argument covers message passing networks whose input is the AIG with node-type features, which is the setting of Section 4.3 and of Zhu et al. [51]. It does not cover higher-order architectures, models given node identifiers or positional encodings, nor encoders whose node features are not a function of the graph alone—the simulation-derived features used by some AIG-specific models are an example, and for those the bound of Figure 3 is not binding.

5.2.3 Empirical performances of AIG-specific models on value prediction

The bounds of Section 5.2.2 say what a representation *could* express. This section asks what models actually trained on the same datasets achieve, which is the quantity a critic in Algorithm 1 would reach.

Models. We train eight regressors on the datasets of Section 5.2.1. Four are generic message passing networks—GCN [18], GIN [47], GAT [43] and GraphSAGE [14]—taken from `PyTorch Geometric` [9]. Three are AIG-specific: DeepGate [19], PolarGate [22] and FuncGNN [49], each run from its authors' implementation. The eighth is the MLP of Section 4.2, which reads x_G and no graph at all, and is the baseline the graph models have to beat. All eight share the same head so that only the encoder differs: node embeddings are mean-pooled into one vector per AIG, which a two-layer ReLU network maps to a scalar. GCN, GIN and GraphSAGE consume the six-way one-hot node type of Section 4.3; GAT, PolarGate and FuncGNN consume the three-way type (input, AND, inverter) of the DeepGate reader, with the inverter bit exposed as a signed edge feature, so that the models that were designed to use complementation receive it. DeepGate is the one exception to end-to-end training: we load the pretrained encoder, freeze it, and train only the head on its mean-pooled functional embeddings, which is how it is meant to be used as a general-purpose circuit encoder.

Training and selection. Targets are $\widehat{V}$ divided by the AND count of the benchmark's initial AIG, so they lie in $(0, 1]$ and are comparable across circuits. Each dataset of $1{,}000$ states is split $80/10/10$ into training, validation and test sets by a permutation seeded by the run seed. Models are trained for $1{,}000$ epochs with Adam [17] on the mean squared error. Each architecture is given its own grid of 36 configurations, and 12 for DeepGate, whose frozen encoder leaves only the head to tune. The grid is run at seed 0, the configuration with the lowest final validation error is selected, and that configuration alone is then rerun on the ten seeds we report; the seed selects the dataset, the split and the initialization together, so the ten repetitions are independent in all three. Appendix A.2 gives the grids. We report test error normalized by the variance of the test targets, so that 1.0 is the error of predicting the mean.

Caveats. Two aspects of the protocol should be read with the numbers below. First, the baseline is the same throughout the paper: the MLP reads x_G, action histogram included, both here and in the bound of Section 5.2.2, and every model is evaluated on the same states and the same targets, so the graph models are measured against one fixed reference rather than against a moving one. Second, the grid is searched at seed 0 only, and the selected configuration is then rerun on ten seeds; a configuration that happens to suit seed 0 could be favored, and this concerns the models trained end to end rather than DeepGate, whose pretrained encoder is frozen and whose grid therefore only tunes a two-layer head. More generally, one grid, one head architecture and $1{,}000$ epochs is less tuning than these encoders received in the papers that introduced them, so we read the result below as evidence that they do not fit this target out of the box, not as an upper bound on what they could reach with more effort.

Trained graph models, including AIG-specific ones, do not beat predicting the mean (Figure 4). All seven graph models land between 0.90 and 1.74 times the target variance — at or above the constant predictor — while the MLP reaches 0.47 at the median. Their training error is also ≈ 1, so this is a failure to fit rather than to generalize, and it leaves them two to three orders of magnitude above the bound of Figure 3 that their own architecture class admits. The per-dataset validation curves of Figure 9 in Appendix D show the graph models settling at the level of the constant predictor early in training and staying there.

6 Future work

Our results are restricted to the setting used by past work: a policy gradient algorithm, five `ABC` primitives as actions, and the fixed horizon of Section 4.1. Two of those choices—the algorithm and

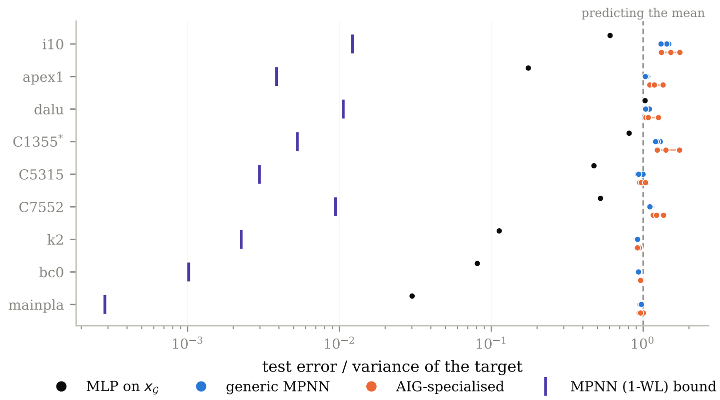

Figure 4: Test error of eight trained models, normalized by the variance of the target so that 1.0 is the error of predicting the mean. Violet ticks: the 1-WL bound of Figure 3.

the action space—bear directly on how much a learned representation of the circuit could contribute, and relaxing them is where we would look next.

Other reinforcement learning algorithms. AIG editing is deterministic and its exploration problem is hard, a combination for which value-based algorithms [45, 27] are usually competitive, yet the existing work on AIG reduction surveyed in Section 2 is entirely policy gradient-based. That distinction matters here rather than only for sample efficiency: a value-based agent acts by comparing the predicted values of successor states, so its behavior rests entirely on the quantity Section 5.2 measures, whereas in an actor-critic method the value estimate only shapes the advantage. If the representation is the bottleneck anywhere, it should be there, and our experiments do not answer whether a graph encoder helps under such an algorithm.

The full `ABC` action space. Five primitives give the policy little to decide, and the states it can reach are the images of a handful of short sequences, which may be why the statistics of Section 4.2 suffice to tell them apart. `ABC` exposes many more primitives, and our environment already implements that larger action space (Section 4.1) even though the experiments above do not use it. With more primitives, reachable states would differ from one another in more ways, and a representation that sees the circuit rather than its summary would have correspondingly more to distinguish. Rerunning both the ablation of Section 5.1 and the bounds of Section 5.2.2 over that action space would say whether our negative result is a property of AIG reduction itself or of the small action space it is usually given.

Further out. Two directions follow from the material above. A finer-grained action space—edits of individual nodes or cones rather than the whole-graph primitives of Figure 1—would produce states that differ locally, and is the setting in which a structural representation would have the most room to help, at the price of having to enforce functional equivalence explicitly rather than inheriting it from the primitives. Separately, Stoll et al. [39] report that DAG generative models fail at AIG reduction without establishing why; the bounds we compute measure what a representation can distinguish, and applying them to the generative setting would test whether the same limitation is at play.

References

References

[1] Eren Akbiyik, Florian Grötschla, Béni Egressy, and Roger Wattenhofer. Graphtester: Exploring Theoretical Boundaries of GNNs on Graph Datasets. In *Data-centric Machine Learning Research (DMLR) Workshop at ICML 2023, Honolulu, Hawaii*, July 2023.

[2] Eric Allender and Bireswar Das. Zero knowledge and circuit minimization. *Information and Computation*, 256:2–8, 2017. ISSN 0890-5401. doi: https://doi.org/10.1016/j.ic.2017.04.004. URL https://www.sciencedirect.com/science/article/pii/S0890540117300512.

[3] Animesh Basak Chowdhury, Marco Romanelli, Benjamin Tan, Ramesh Karri, and Siddharth Garg. Retrieval-guided reinforcement learning for boolean circuit minimization. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 35039–35060, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/96bbdd0ed2a9e7cd2fb7caf2fae15f3d-Paper-Conference.pdf.

[4] Richard Bellman. *Dynamic Programming*. 1957.

[5] Robert K. Brayton and Alan Mishchenko. Abc: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*, 2010. URL https://api.semanticscholar.org/CorpusID:1251520.

[6] Alba Carballo-Castro, Manuel Madeira, Yiming QIN, Dorina Thanou, and Pascal Frossard. Generating directed graphs with dual attention and asymmetric encoding. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=s2s5xGQCKM.

[7] Andrea Costamagna, Alan Mishchenko, Satrajit Chatterjee, and Giovanni De Micheli. An enhanced resubstitution algorithm for area-oriented logic optimization. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2024. doi: 10.1109/ISCAS58744.2024.10558264.

[8] Victor-Alexandru Darvariu, Stephen Hailes, and Mirco Musolesi. Graph reinforcement learning for combinatorial optimization: A survey and unifying perspective. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=HduK51xNtS. Survey Certification.

[9] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[10] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017.

[11] Antoine Grosnit, Cedric Malherbe, Rasul Tutunov, Xingchen Wan, Jun Wang, and Haitham Bou Ammar. Boils: Bayesian optimisation for logic synthesis. In *Design, Automation, Test in Europe Conference, 2022*, pages 1193–1196, 2022. doi: 10.23919/DATE54114.2022.9774632.

[12] Winston Haaswijk, Edo Collins, Benoit Seguin, Mathias Soeken, Frédéric Kaplan, Sabine Süsstrunk, and Giovanni De Micheli. Deep learning for logic optimization algorithms. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–4, 2018. doi: 10.1109/ISCAS.2018.8351885.

[13] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.

[14] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[15] Abdelrahman Hosny, Soheil Hashemi, Mohamed Shalan, and Sherief Reda. Drills: Deep reinforcement learning for logic synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 581–586, 2020. doi: 10.1109/ASP-DAC47756.2020.9045559.

[16] Valentine Kabanets and Jin-Yi Cai. Circuit minimization problem. In *Proceedings of the thirty-second annual ACM symposium on Theory of computing*, pages 73–79, 2000.

[17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[18] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[19] Min Li, Sadaf Khan, Zhengyuan Shi, Naixing Wang, Huang Yu, and Qiang Xu. Deepgate: Learning neural representations of logic gates. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 667–672, 2022.

[20] Mufei Li, Viraj Shitole, Eli Chien, Changhai Man, Zhaodong Wang, Srinivas, Ying Zhang, Tushar Krishna, and Pan Li. LayerDAG: A layerwise autoregressive diffusion model of directed acyclic graphs for system. In *Machine Learning for Computer Architecture and Systems 2024*, 2024. URL `https://openreview.net/forum?id=IsarrieeQA`.

[21] Fangzhou Liu, Wuqian Tang, Zehua Pei, Ziyang Yu, Haisheng Zheng, Zhuolun He, Mengjia Dai, Tinghuan Chen, and Bei Yu. Cb-evo: Contextual bandit tuning with evolutionary search for logic synthesis. *ACM Trans. Des. Autom. Electron. Syst.*, 31(6), June 2026. ISSN 1084-4309. doi: 10.1145/3779431. URL `https://doi.org/10.1145/3779431`.

[22] Jiawei Liu, Jianwang Zhai, Mingyu Zhao, Zhe Lin, Bei Yu, and Chuan Shi. Polargate: Breaking the functionality representation bottleneck of and-inverter graph neural network. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024.

[23] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim M. Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, Will Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chip design. *Nature*, 594: 207 – 212, 2021. URL `https://api.semanticscholar.org/CorpusID:235395490`.

[24] A. Mishchenko, S. Chatterjee, and R. Brayton. Dag-aware aig rewriting: a fresh look at combinational logic synthesis. In *2006 43rd ACM/IEEE Design Automation Conference*, pages 532–535, 2006. doi: 10.1145/1146909.1147048.

[25] Alan Mishchenko and Robert K. Brayton. Scalable logic synthesis using a simple circuit structure. 2006. URL `https://api.semanticscholar.org/CorpusID:8597391`.

[26] Alan Mishchenko, Satrajit Chatterjee, Roland Jiang, and Robert K. Brayton. Fraigs: A unifying representation for logic synthesis and verification, 2005. URL `https://api.semanticscholar.org/CorpusID:9209313`.

[27] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[28] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR. URL `https://proceedings.mlr.press/v48/mniha16.html`.

[29] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4602–4609, 2019.

[30] Walter Lau Neto, Yingjie Li, Pierre-Emmanuel Gaillardon, and Cunxi Yu. Flowtune: End-to-end automatic logic optimization exploration via domain-specific multiarmed bandit. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(6):1912–1925, 2023. doi: 10.1109/TCAD.2022.3213611.

[31] Suhua Ou, Qingshan Yang, and Jian Liu. The global production pattern of the semiconductor industry: an empirical research based on trade network. *Humanities and Social Sciences Communications*, 11(1):1–13, 2024.

[32] Rajendran Panda and Farid N. Najm. Post-mapping transformations for low-power synthesis. *VLSI Design*, 7:289–301, 1998. URL https://api.semanticscholar.org/CorpusID:6811282.

[33] Zehua Pei, Fangzhou Liu, Zhuolun He, Guojin Chen, Haisheng Zheng, Keren Zhu, and Bei Yu. Alphasyn: Logic synthesis optimization with efficient monte carlo tree search. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9, 2023. doi: 10.1109/ICCAD57390.2023.10323856.

[34] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[35] Yu Qian, Xuegong Zhou, Hao Zhou, and Lingli Wang. An efficient reinforcement learning based framework for exploring logic synthesis. *ACM Trans. Des. Autom. Electron. Syst.*, 29(2), January 2024. ISSN 1084-4309. doi: 10.1145/3632174. URL https://doi.org/10.1145/3632174.

[36] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[37] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[38] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and Philip S Yu. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*, 29(1): 17–37, 2016.

[39] Timo Stoll, Chendi Qian, Ben Finkelshtein, Ali Parviz, Darius Weber, Fabrizio Frasca, Hadar Shavit, Antoine Siraudin, Arman Mielke, Marie Anastacio, Erik Müller, Maya Bechler-Speicher, Michael Bronstein, Mikhail Galkin, Holger Hoos, Mathias Niepert, Bryan Perozzi, Jan Tönshoff, and Christopher Morris. Graphbench: Next-generation graph learning benchmarking, 2026. URL https://arxiv.org/abs/2512.04475.

[40] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018. ISBN 0262039249.

[41] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

[42] Gerald Tesauro. Temporal difference learning and td-gammon. *Commun. ACM*, 38(3):58–68, March 1995. ISSN 0001-0782. doi: 10.1145/203330.203343. URL https://doi.org/10.1145/203330.203343.

[43] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[44] Hanrui Wang, Kuan Wang, Jiacheng Yang, Linxiao Shen, Nan Sun, Hae-Seung Lee, and Song Han. Gcn-rl circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2020. doi: 10.1109/DAC18072.2020.9218757.

[45] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.

[46] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8(3–4):229–256, May 1992. ISSN 0885-6125. doi: 10.1007/BF00992696. URL https://doi.org/10.1007/BF00992696.

[47] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[48] Wenlong Yang, Lingli Wang, and Alan Mishchenko. Lazy man's logic synthesis. In *Proceedings of the International Conference on Computer-Aided Design*, ICCAD '12, page 597–604, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450315739. doi: 10.1145/2429384.2429513. URL `https://doi.org/10.1145/2429384.2429513`.

[49] Qiyun Zhao. Funcgnn: Learning functional semantics of logic circuits with graph neural networks, 2025.

[50] Ziyang Zheng, Shan Huang, Jianyuan Zhong, Zhengyuan Shi, Guohao Dai, Ningyi Xu, and Qiang Xu. Deepgate4: Efficient and effective representation learning for circuit design at scale. In *The Thirteenth International Conference on Learning Representations*, 2025. URL `https://openreview.net/forum?id=b10lRabU9W`.

[51] Keren Zhu, Mingjie Liu, Hao Chen, Zheng Zhao, and David Z. Pan. Exploring logic optimizations with reinforcement learning and graph convolutional network. In *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)*, pages 145–150, 2020. doi: 10.1145/3380446.3430622.

A Hyper-parameters

A.1 Graph encoders and reinforcement learning

Table 1 lists the four graph encoders of Section 4.3. The baseline uses no encoder at all and reads $x_{\mathcal{G}}$ alone.

Table 1: The four graph encoders. Depth is the number of message passing rounds, width the hidden dimension, and output the dimension of the pooled vector concatenated with $x_{\mathcal{G}}$.

encoder	depth	width	output
`small`	3	12	4
`medium`	3	64	32
`large`	3	128	64
`v. large`	5	128	64

Policies are trained with the PPO implementation of `stable-baselines3` [36], as stated in Section 4.4. Eight environments run in parallel and each contributes 20 steps per update, so an update is computed from 160 transitions and a single episode spans five updates; $8{,}000$ environment steps therefore amount to $N = 50$ updates. The policy and value heads are each two hidden layers of 256 units and share the encoder of line 7 of Algorithm 1.

Algorithm 1 GraphPPO. The encoder f_θ is the only component that varies across the runs we compare; everything else is held fixed.

Require: AIG $\mathcal{G}_0$ to reduce with n_0 AND nodes, encoder f_θ, policy head π_θ, value head V_θ, action set A of five `ABC` primitives, fixed horizon $H = 20$, iterations N, epochs K, clipping ϵ, entropy weight β, value weight c, step size η

1: $n^\star \leftarrow n_0$
2: **for** $i = 1, \ldots, N$ **do**
3: $\quad \mathcal{D} \leftarrow \emptyset$
4: $\quad$ **for** each parallel environment **do**
5: $\quad\quad \mathcal{G} \leftarrow \mathcal{G}_0$
6: $\quad\quad$ **for** $t = 0, \ldots, H-1$ **do**
7: $\quad\quad\quad s_t \leftarrow \big(f_\theta(\mathcal{G}),\ x_{\mathcal{G}}\big)$ ▷ f_θ is a GCN, or is absent for the baseline
8: $\quad\quad\quad a_t \sim \pi_\theta(\cdot \mid s_t), \quad a_t \in A$
9: $\quad\quad\quad \mathcal{G} \leftarrow \textsc{Abc}(\mathcal{G}, a_t)$ ▷ deterministic, preserves functional equivalence
10: $\quad\quad\quad r_t \leftarrow 1 - |V^{AND}(\mathcal{G})| \,/\, n_0$ ▷ relative size, issued at every step
11: $\quad\quad\quad \mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, \log \pi_\theta(a_t \mid s_t), V_\theta(s_t))\}$
12: $\quad\quad\quad n^\star \leftarrow \min\big(n^\star,\ |V^{AND}(\mathcal{G})|\big)$
13: $\quad\quad$ **end for**
14: $\quad$ **end for**
15: $\quad$ compute returns $\hat{R}_t$ and advantages $\hat{A}_t$ from $\mathcal{D}$
16: $\quad \theta_{\text{old}} \leftarrow \theta$
17: $\quad$ **for** K epochs over minibatches of $\mathcal{D}$ **do**
18: $\quad\quad \rho_t(\theta) \leftarrow \pi_\theta(a_t \mid s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$
19: $\quad\quad L(\theta) \leftarrow \widehat{\mathbb{E}}_t\big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big) - c\big(V_\theta(s_t) - \hat{R}_t\big)^2 + \beta\,\mathcal{H}\big[\pi_\theta(\cdot \mid s_t)\big]\big]$
20: $\quad\quad \theta \leftarrow \theta + \eta\,\nabla_\theta L(\theta)$ ▷ gradients flow through f_θ
21: $\quad$ **end for**
22: **end for**
23: **return** $n^\star$, the smallest AND count visited

The grid of Section 5.1 crosses three learning rates (10^{-4}, 10^{-5}, 10^{-6}), three entropy coefficients (0, 0.01, 0.1) and five seeds, which with five encoders and ten benchmarks gives the 2,250 runs reported there. Twenty-six of those runs (eighteen `small`, eight `v. large`) stopped before the fiftieth update; we drop them rather than pad them, and requiring all five encoders to have completed a given (benchmark, learning rate, entropy, seed) cell leaves 424 of the 450 cells.

A.2 Value prediction

The models of Section 5.2.3 are trained in mini-batches of 16 unless the batch size is itself searched over. Each architecture's grid of 36 configurations crosses three learning rates (10^{-3}, 5×10^{-4}, 10^{-4}) with a width and a depth, or with the number of attention heads for GAT and the polarity weight for PolarGate. DeepGate is the exception, with 12 configurations: its pretrained encoder is frozen, so only the two-layer head is tuned.

B The ex200–ex299 suite

We repeat the comparison of Section 5.1 on ex200–ex299, a suite spanning 106 to 46,977 input AND gates and therefore both larger and more varied than the ten MLCAD circuits. Each encoder ran at its own baseline hyper-parameters here instead of over the matched grid. The three GCNs reduce the final AND count by 0.33–0.71% against a reseed standard deviation of 0.54% (Figure 5), and the difference is flat across two and a half orders of magnitude of circuit size (Figure 6), so the conclusion of Section 5.1 appears to hold here as well. The MLP runs were executed on CPU and did not finish on the largest circuits, which is why 20 of the 100 benchmarks are excluded; the comparison is therefore made on the smaller eighty, and we cannot rule out that the largest circuits behave differently.

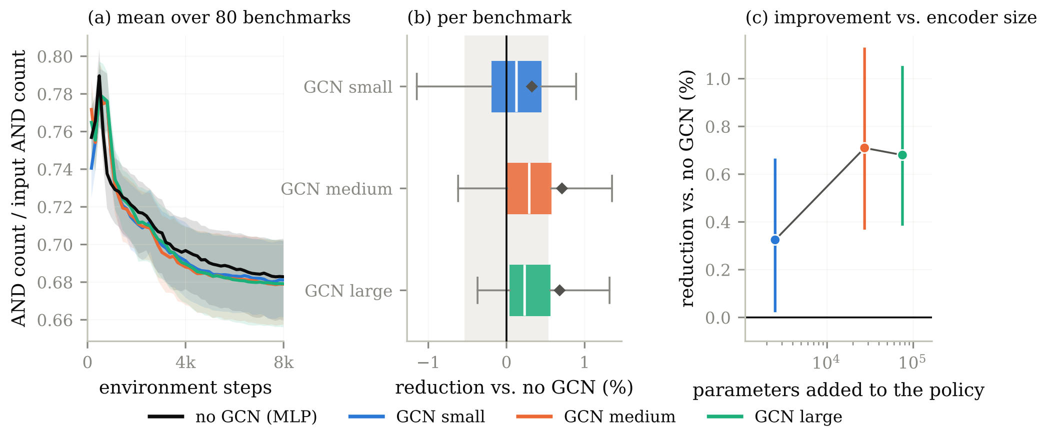

Figure 5: The comparison of Figure 2 on ex200–ex299, where each encoder ran at its own baseline hyper-parameters instead of the matched grid. Restricted to the 390 of 500 (benchmark, seed) cells all four encoders completed, covering 80 benchmarks.

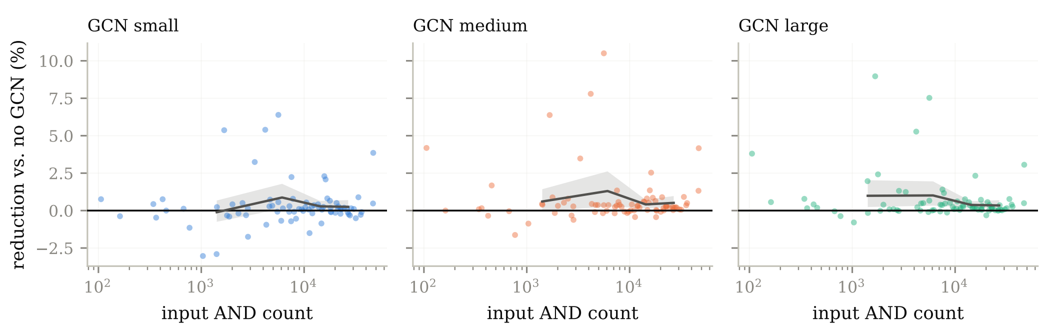

Figure 6: Reduction against the MLP as a function of input circuit size on ex200–ex299. Line: mean over size quartiles.

C The message passing bound against depth

Figure 7 plots the message passing bound of Section 5.2.2 against the number of refinement iterations k, for the four views of the AIG bounded there. The bound moves by at most 1.1×10^{-3} between $k = 1$ and $k = 50$, and labeling edges with direction or with the complement bit changes it by less than 10^{-6}, so neither deeper refinement nor a richer view of the graph brings a message passing network closer to $\widehat{V}$.

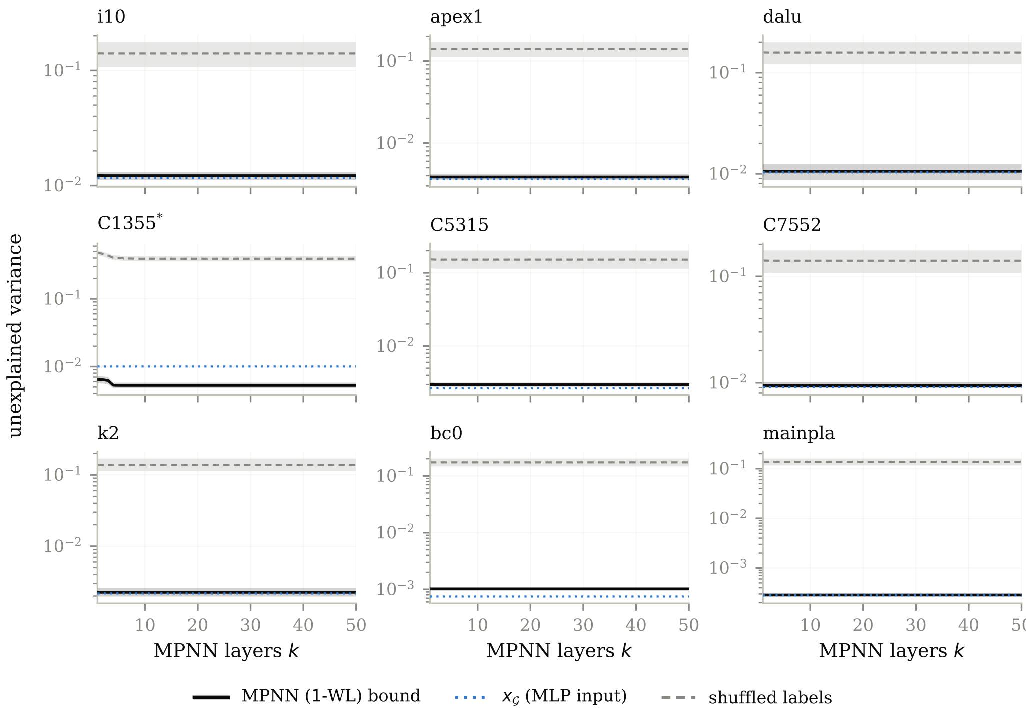

Figure 7: The MPNN bound as a function of depth k.

D Per-dataset curves

Figure 8 resolves the aggregate of Figure 2(a) into one learning curve per benchmark, and Figure 9 shows validation error over training for the value prediction models of Figure 4.

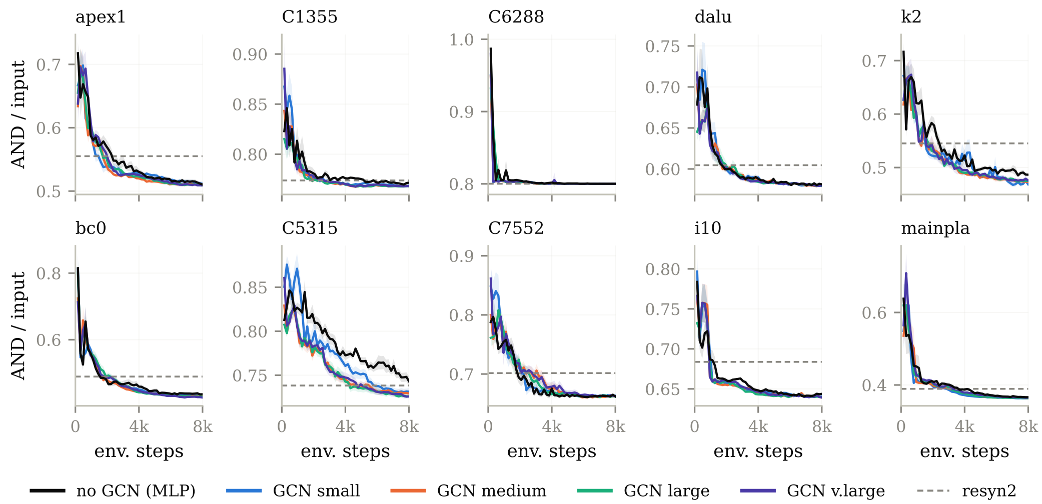

Figure 8: Per-benchmark learning curves for the runs aggregated in Figure 2(a). Dashed line: `resyn2`.

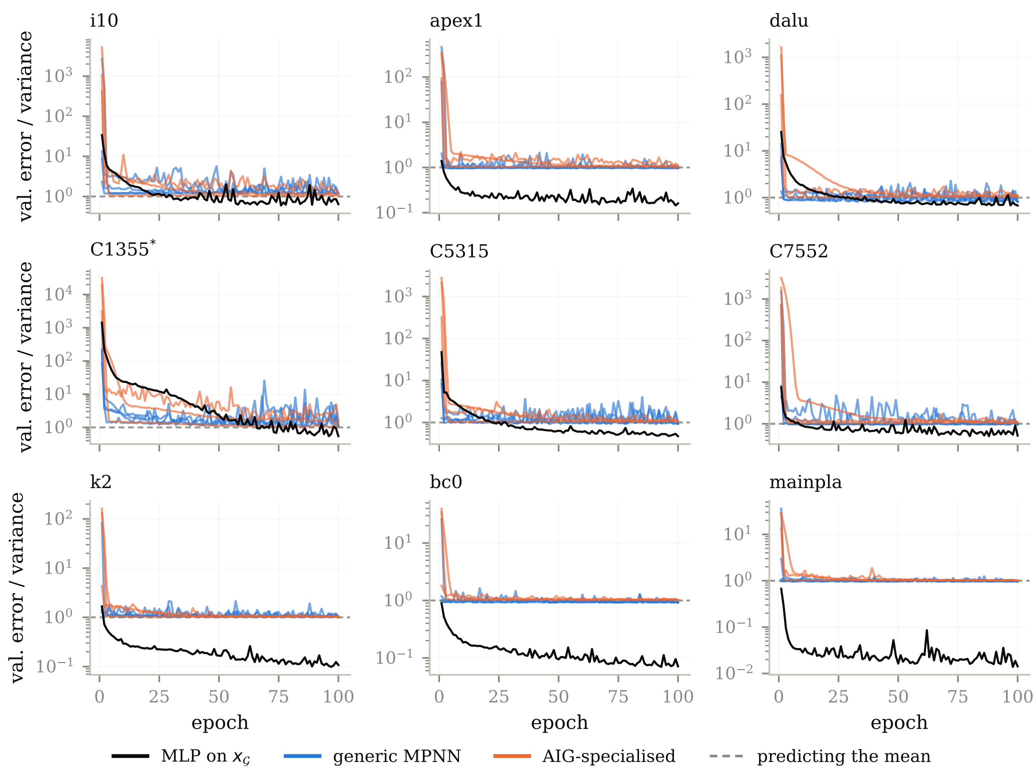

Figure 9: Validation error over training for the configurations of Figure 4, one panel per benchmark.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.